Questionário - TodoList2

Projeto: TodoList2 (Flutter) | github.com/JeanLucas05/TodoList2

1. Estrutura do projeto

Projeto Flutter (Android + Linux desktop), com todo o código em `lib/`:

- **Telas:** `lib/screens/` (`task_list_screen.dart`, `task_editor_screen.dart`, `category_management_screen.dart`)
- **Modelos:** `lib/models/` (`Task`, `Category`: imutáveis, com `copyWith`, `toMap` e `fromMap`)
- **SQLite:** `lib/data/` (`app_database.dart`, `task_repository.dart`, `category_repository.dart`)
- **Navegação:** `lib/app.dart` (`MaterialApp.routes` e `onGenerateRoute`)
- **Notificações:** `lib/services/notification_service.dart`
- **Estado:** `lib/state/todo_controller.dart`; `lib/main.dart` faz o bootstrap

2. Arquitetura e estado

- Usa o pacote `provider` com **um único** `ChangeNotifier`, o `TodoController`. Ele guarda `_tasks`, `_categories`, `statusFilter`, `categoryFilterId`, `isLoading` e `loadError`.
- É a arquitetura **repository + controller**: os repositórios têm todo o SQL, o controller orquestra e as telas só falam com o controller.
- Depois de criar, editar ou concluir uma tarefa, o controller chama `_reloadAfterMutation()`, que relê as duas tabelas sem ligar `isLoading` (a lista não pisca vazia). Como as telas escutam o `ChangeNotifier`, elas se reconstroem sozinhas.
- Toda mutação retorna `Future<String?>` (null = sucesso); a tela mostra um `SnackBar` com a mensagem de erro.
- Os filtros rodam em memória, em `filteredTasks`.

3. Persistência SQLite

- **Criação:** `AppDatabase` é dona da conexão e do schema (versão 1, `onCreate`, `PRAGMA foreign_keys = ON`). O arquivo `todoapp.db` fica em `getApplicationSupportDirectory()`.
- **Armazenamento:** `categories(id, name, color)` e `tasks(id, title, description, completed, dueDateTime, createdAt, categoryId)`. Datas em epoch (milissegundos); `dueDateTime` nulo = sem prazo. `categoryId` tem `ON DELETE SET NULL`.
- **CRUD:** em `TaskRepository` e `CategoryRepository`, que lançam `DatabaseException`; o controller converte a exceção em mensagem para o usuário.
- **Bibliotecas:** `sqflite` no Android e `sqflite_common_ffi` no desktop.

4. Fluxo de criar uma tarefa

Os nomes exatos dos métodos abaixo são inferidos do log.

1. **Save** no `TaskEditorScreen` valida o formulário e chama o `TodoController.createTask`.
2. O controller chama o `TaskRepository.insert`, que grava no SQLite e **retorna o id gerado**.
3. O controller chama `syncForTask` do `NotificationService`.
4. O controller chama `_reloadAfterMutation()` e o `ChangeNotifier` avisa as telas. A lista mostra a nova tarefa e o editor fecha.

5 O `syncForTask` cancela qualquer notificação com o mesmo id. Depois só agenda com `zonedSchedule` se a tarefa tiver id, estiver pendente e tiver prazo no futuro.

5. Navegação

- `Navigator.pushNamed`. A lista e as categorias usam `MaterialApp.routes`; o editor usa `onGenerateRoute`, porque precisa de argumento.
- Só o **id da tarefa** é passado (`TaskEditorScreen.idOf(settings.arguments)`). O editor busca a tarefa com `TodoController.taskById`, então nunca fica com uma cópia desatualizada.

6. Notificações

- **Agendamento:** `flutter_local_notifications` com `zonedSchedule`, usando `TZDateTime` no fuso do aparelho (`flutter_timezone`). Tenta `exactAllowWhileIdle` e, se der `exact_alarms_not_permitted`, refaz com `inexactAllowWhileIdle`.
- **Associação:** o **id da notificação é o id da tarefa**; o payload é `task:<id>`.
- **Prazo alterado:** o `syncForTask` cancela a antiga e reagenda a nova.
- **Concluída, deletada ou prazo removido:** ele cancela e não reagenda, porque só agenda tarefas pendentes com prazo futuro.
- Falhas (`UnimplementedError` no Linux, por exemplo) são capturadas e logadas, sem derrubar o app.

7. Decisões do agente que a especificação não definia

- `sqflite` **em vez de** `drift`**:** sem geração de código, com SQL explícito e poucas dependências.
- `provider` **com um controller único:** Riverpod e Bloc foram descartados por serem complexos demais. Dois notifiers dependeriam um do outro, já que a lista precisa dos nomes das categorias.
- Outras: o id da tarefa serve de id da notificação, os filtros rodam em memória, e existe uma camada `NotificationsApi` para testar o serviço. O Linux desktop entrou como alvo de validação.

8. Análise do BUILD_LOG

Problema (Entrada #10)

O `flutter build apk` falhou com “No Android SDK found. Try setting the ANDROID_HOME environment variable”, mas o `ANDROID_HOME` estava certo.

Primeira solução

O agente diagnosticou antes de pedir qualquer coisa e mandou os comandos de instalação. Eles não tinham funcionado antes: o `apt install openjdk-17-jdk` nunca rodou, então não havia Java, o `sdkmanager` morria e o SDK ficou sem `platform-tools/` nem `licenses/`. A checagem `validSdkDirectory` do Flutter falhava, e a mensagem enganava.

Solução final

O agente não tem `sudo`. O usuário instalou JDK 17 e o SDK em outro terminal. O build passou (APK debug e release), o manifesto foi verificado, e o app foi instalado num Infinix pelo `adb.exe` do Windows, com caminho no formato Windows.

O que o log mostra e o código final não mostra

- **Por que essas escolhas:** as alternativas rejeitadas (drift, Riverpod, Bloc).

- **Cancelar e reagendar:** a decisão de sempre cancelar e depois reagendar.
- **Controller e serviço:** o teste que falhou na Entrada #6 mostra que quem decide agendar é o serviço, não o controller.
- **Limites da validação:** no Linux não dá para agendar notificação. No celular, o log só mostra o pedido de permissão na tela, sem nenhuma notificação disparando.
- **Honestidade do registro:** a Entrada #9 corrige um erro do próprio log, que dizia que já havia commits.